ChatGPT

Critical Performance Evaluation of the Omoknuni MCTS Engine

Hardware & Goals: The MCTS engine is intended to achieve **30,000+ simulations/sec** on an AMD Ryzen 9 5900X (12 cores/24 threads) with a mid-range GPU (RTX 3060 Ti) [1]. In practice, the current implementation reaches only ~3,000–3,800 sims/sec [2] – an order of magnitude below target. Below, we pinpoint **why throughput is so low**, examining the C++ backend and Python orchestration in detail, then propose a concrete plan to reach the throughput goal. The analysis is **blunt**: every inefficiency or design flaw is called out, and major changes are suggested if necessary.

Identified Bottlenecks in the Current MCTS Implementation

1. Excessive Python Involvement in the Hot Loop

Despite a design goal that "**Python never touches hot loops**" [3], the current engine still incurs significant Python overhead during search. The **core search loop** is not fully in C++ – Python does part of the work for each simulation. For example, in older code, `AlphaZeroMCTS.search()` iterated in Python, cloning game states and applying moves on each step [4] [5]. This violated the shared-tree design and GIL-free ideal, effectively running simulations single-threaded and capping performance around ~1k sims/sec [6] [7].

Current status: The repository has since introduced a C++ `SimulationRunner` and `ContinuousSimulationRunner` with GIL released (via `py::call_guard<py::gil_scoped_release>`) so that the heavy loop runs in C++ across threads. This is a **big improvement**. However, **Python still intervenes in critical ways**:

- **GameState cloning & moves:** Each simulation clones the game state (twice in async mode) and applies moves. These calls ultimately invoke C++ (`IGameState.clone()` and `makeMove()` in the game logic), but they are triggered from Python or pybind wrappers. There's per-call overhead crossing Python/C++ boundaries. We do see Gomoku's state is optimized in C++ (bitboard representation, cached legal moves, etc.), which is good for move generation. But still, **each simulation spawns multiple Python↔C++ interactions**. Even after moving the main loop to C++, **the Python layer handles batching of inference inputs and result callbacks**, which adds latency. The `AlphaZeroMCTS._create_batch_inference_callback()` shows that for each batch the code iterates in Python to convert game states to NumPy arrays and then calls the neural net [8] [9]. These Python conversions and list operations are happening thousands of times per second, stealing cycles from the CPU that could run more simulations.

- **Neural network interface:** The MCTS uses a Python-based neural network (PyTorch) for value/policy. Even with asynchronous batching, the coordinator thread is calling a Python callback for each batch, assembling tensors in Python. This "Python in the loop" overhead is non-trivial. In fact, project documentation admits an optimized *C++ libtorch* inference was deliberately not used to keep NN modification easy – but the trade-off is higher overhead per inference. The current code tries to

mitigate this by providing a batch API (so one Python call can evaluate e.g. 32 states at once), but the conversion of states to tensors is done in Python, and if the `inference_fn` isn't a `GPUInferenceWorker` with a `batch_inference` method, it falls back to a **slow per-state mode (~1k sims/sec)** [10] [11]. In short, if the user isn't using the optimized batch path, the performance plummets. Even in "fast" mode, Python overhead is present in batching.

- **ThreadPoolExecutor orchestration:** The Python `ThreadPoolExecutor` is used to spawn simulation threads for a search [12]. There is some overhead in submitting tasks and collecting futures for each search. The code does reuse the executor across searches (it caches it in `self._executor`), which is good, but starting and stopping the *BatchInferenceCoordinator* each search still involves Python calls and thread startup/teardown every time [13] [14]. This adds latency especially for small searches.

Bottom line: While much of the heavy lifting is in C++ now, **the Python layer still contributes an estimated 60–70% of runtime overhead** [2] through coordination costs. Every simulation triggers Python-managed operations (batching, state conversion), preventing the engine from fully utilizing the CPU. To hit 30k sims/sec, Python's role must be minimized further.

2. Inefficient Asynchronous Inference Coordination

The MCTS uses an asynchronous approach: simulation threads push inference requests to a queue and never wait for the neural net (they continue with other simulations) [15]. This design is correct in principle for throughput. However, the **implementation of the producer/consumer pipeline is inefficient**, leading to contention and wasted CPU cycles:

- **Busy-wait polling:** The `ContinuousSimulationRunner` loop uses a polling mechanism to check for completed NN results. If no result is ready, threads sleep for a very short interval (50–100 µs) and try again [16]. Similarly, if the queue is full (`>=4096` pending), a thread will spin in a tight loop, repeatedly calling `process_completed_results()` and sleeping in microseconds until a slot frees [17] [18]. These frequent wakes add CPU overhead – threads spend significant time in spin-wait, which **burns CPU time that could run more simulations**. There is no use of condition variables or proper blocking notifications in the current queue; it's effectively a manual spinlock with sleeps.

- **Lock contention on queue and map:** The `AsyncInferenceQueue` and the global `pending_expansions_` map in `ContinuousSimulationRunner` introduce synchronization overhead. Each inference request is assigned a `request_id` and stored in an `unordered_map<uint64_t, PendingExpansion>` for later lookup [19] [20]. A global lock likely guards the queue operations (or atomic operations on its counters), and accessing the `unordered_map` for every result involves locking its buckets and potential rehash costs. At 30k sims/sec, there could be many thousands of queue operations per second; the current approach (push/pop one request at a time, map insert/erase per request) does not scale well. The code itself notes these overheads: the spec proposes an **MPMC ring buffer and batched pops** to replace the current queue and eliminate per-result map lookups [21] [22].

- **Pending result handling overhead:** When a batch of results arrives, the coordinator thread (in C++) calls `queue.consume_ready_results()` to get a vector of results, then each result is processed

in a loop, doing a map lookup and erase for each [23] [20]. This is O(n) per result batch, and uses `unordered_map` which is not cache-friendly for large numbers of entries. If many simulations are in flight, this map could grow large, and lookups/erasures become costly. The **map also forces us to reverse the path vector for backup each time** (the code does `std::reverse(path.begin(), path.end())` on every result [24]). Reversing a short vector isn't huge, but it's an extra step that could be avoided if the tree stored parent links or if backup could handle forward order. All these small costs add up given the scale of operations.

Impact: These coordination inefficiencies mean **67% of runtime is spent outside actual neural net inference** [2]. The CPU isn't fully busy doing game tree expansion; it's busy managing threads, waiting on the queue, and shuffling data structures. This explains why on a 12-core CPU, we only see ~3k sims/sec – the threads are often idle or doing bookkeeping instead of traversing the tree.

3. Suboptimal Multi-Thread Utilization and Contention

The code is intended to scale up to 8–12 threads searching one tree concurrently [25]. However, a few issues prevent full utilization of all CPU cores:

- **Virtual loss contention at root:** In a shared tree MCTS, all threads start from the root. When the root is unexpanded initially, one thread will "grab" it to expand (using `atomic_try_mark_expanding`), and others detect the flag and back off (`waiting_for_leaf = true`) [26] [27]. During this time, those other threads essentially sleep 50 µs at a time waiting for the root expansion to finish [16]. This means at the very start of search, **(N-1) threads are idle** for the entire neural network inference latency (e.g. a few milliseconds) – a significant sequential bottleneck. The same can happen deeper in the tree: if many threads converge on a single new leaf (e.g. a very promising move), only one can expand it while others wait. In worst-case scenarios (e.g. all threads explore the same principal variation), virtual loss helps avoid outright duplication, but threads still serialize at each new frontier node. This limits parallel efficiency.

- **Cache coherence and atomic overhead:** The shared memory Structure-of-Arrays tree is highly optimized, but with many threads updating it, there is contention on atomic updates to node statistics. For example, every backup does atomic `visit_count++` and `total_value+=v` along the path [28] [29]. These are fine-grained atomics and can cause cache line ping-pong between cores. The guide notes that cross-CCD (chiplet) latency on the 5900X is ~80ns vs 20ns intra-CCD [30]. If threads are not affinitized, two threads working on the same section of the tree but on different CCDs will incur additional latency on every atomic op. The more threads, the more cache coherence traffic. Scaling to 12 threads might yield diminishing returns if most updates target the upper tree levels (which they often do early on). **This could partially explain why adding threads hasn't scaled throughput linearly** – beyond a certain count, contention overhead grows.

- **Thread oversubscription in self-play:** While not a factor in a single search, it's worth noting a design issue if multiple searches run concurrently. The review pointed out that the `SelfPlayCoordinator` could spawn multiple `AlphaZeroMCTS` instances each with their own thread pool, leading to an explosion of threads (N games * MCTS threads per game) and "the host will thrash under load" [31]. If you were running, say, 10 simultaneous self-play games with the default 8 threads each, that's 80 threads contending – far exceeding 24 hardware threads. This oversubscription would tank performance. The fix would be to **reuse a shared thread pool or limit**

threads per MCTS when running many in parallel [31]. It's unclear if the current code addresses this (no evidence of a global pool in the snippet), so it's a concern when scaling training.

- **No thread affinity or NUMA awareness:** The code does not pin threads to specific cores or NUMA nodes. On Linux, threads might migrate and compete for CPU caches unpredictably. The 5900X, while a single socket, has two CCX/CCD units; ideally one would pin half the threads to each CCD to keep their memory accesses local to that CCD's L3 cache. Currently, threads are launched without affinity control, so the OS scheduler might bounce them around, causing cross-CCD memory access (especially since the MCTS tree is one large structure likely allocated on one NUMA node). This can hurt performance subtly. A fully optimized engine might allow setting thread affinity (though Python's `ThreadPoolExecutor` doesn't natively do this).

In summary, the multi-threading is conceptually in place (thanks to virtual loss and atomic updates), but *practical inefficiencies* – root expansion serialization, atomic overhead, and potential thread misuse – mean the engine is **not leveraging all 12 cores effectively**. The spec target is a ≥6× speedup from 1→8 threads [32], but currently it's nowhere close.

4. Memory Management and Tree Initialization Costs

The MCTS tree data structure uses a massive pre-allocated array for nodes (default max 10 million nodes = ~270 MB) [33] [34]. This ensures fast indexing and avoids dynamic allocation per node. However, the way this memory is managed per search is costly:

- **Full tree clearing:** On each new search, the code does `self.tree.clear()`, which likely *memsets* or reinitializes the entire node pool to zeros [35]. Indeed, the constructor in the guide explicitly memsets `visit_counts` for `max_nodes` [36]. Clearing 10 million nodes means writing ~270 MB of data. Even if `get_node_count()` from the last search was small, the clear still touches the full buffer. This is a huge overhead if searches are repeated frequently. The review also flags this: *"each call to search() does tree.clear() which memsets the entire memory"* [37]. This alone can take tens of milliseconds or more, eating into throughput. It also blows away CPU cache each time. The spec suggests a smarter strategy, such as **generation counters or segmented clears to avoid bulk memset** [38]. (Indeed, in spec Task 2 it appears they implemented an "epoch-based lazy clear" to avoid this full reset in newer code.)

- **Node allocation contention:** The engine allocates children in contiguous blocks via `tree.allocate_nodes(num_children)`. In the current implementation, this likely does an atomic fetch-add on a global `num_nodes` counter to reserve space for new nodes. Under heavy multi-threading, this counter becomes a contention point if many threads expand nodes simultaneously. The spec proposes **per-thread node arenas or bulk allocation** to reduce allocator contention [38] [39]. Without that, expansions can bottleneck: threads might effectively serialize when many nodes are added at once, since each must atomically update the shared `num_nodes`. This could be limiting the tree growth rate and thus simulation count.

- **Move storage overhead:** (This was an issue in older code, partly mitigated now.) Previously, the mapping from node index to move was stored in a Python dict `_move_mapping`, leading to huge memory overhead for large trees [40] [41]. The current code appears to have moved this into C++: each node's move is stored via `tree.set_move(child_idx, move)` during expansion [42], and

retrieved by `tree.get_move()` in Python when computing the final policy [43] [44]. This is good – it avoids Python dict overhead for moves. However, the Python `get_policy()` still builds a visits array by iterating children and calling `get_move` and `get_visit_count` per child [45]. For a broad branching factor (e.g. 300+ moves in Go 19x19), this is a Python loop of 300 iterations per search, which is fine, but if the root has thousands of children (unlikely in games), that could be slow. Not a primary issue, but mentionable for completeness: critical paths are mostly elsewhere.

- **Memory footprint vs. locality:** Each node is ~32–64 bytes spread across multiple arrays (SoA) [46]. This is compact, but it also means a DFS traversal (selection) will jump around in memory (from one array to another for each node's data). The engine likely relies on caching entire arrays and sequential child placement to keep things somewhat contiguous. If `max_nodes` is huge and only a small portion is used, those arrays still occupy a large memory footprint in the process, possibly affecting cache and TLB. But since it's mostly sequential in memory, this is not a big slowdown factor, just a memory overhead consideration. The key is to ensure we *don't allocate way beyond need* – e.g., if Gomoku rarely needs >1e6 nodes, using 10e6 as default may be overkill and makes clears slower. The code defaults to 10M; perhaps a dynamic sizing or lower default for smaller games could save time.

5. Neural Network Inference Latency & Throughput

Finally, the neural network itself can be the bottleneck if not used optimally:

- **Batching and GPU utilization:** The goal is to batch many leaf evaluations to amortize GPU call overhead and keep the device busy. The design uses `async_batch_size=32` and `async_timeout_ms=2.0` by default [47], aiming for ~32 states per batch or ~2ms latency. In practice, achieving ~80–90% GPU utilization on a RTX 3060 Ti is challenging [48] [49]. If the batch sizes are often smaller (say, during the tail of a search when fewer leaf nodes are being expanded), the GPU may be under-utilized. At ~3k sims/sec, the GPU is likely running far below capacity (given a 3060 Ti can do on the order of 10^4+ inferences/sec for a small network). This indicates either the GPU is waiting on CPU, or batch sizes are suboptimal. **If the GPU is not saturated, we're not extracting full throughput.**

- **Network size and CPU fallback:** The actual neural net architecture isn't described here, but if it's large (e.g. a ResNet-20 or similar), each inference might take a few milliseconds even on GPU. The code enables FP16 autocast and suggests pinned memory for transfers [50], which is good. But if those aren't actually implemented (the `GPUInferenceWorker` might need to handle autocast), there's room to speed up inference by using half precision. Also, if the user attempted to run inference on CPU (since they mentioned wanting to harness the CPU and the GPU being mid-spec), note that would be *much* slower. A CPU-bound Torch model, even multi-threaded, won't hit 30k sims/sec unless the model is extremely tiny, because the CPU would become the bottleneck. The **optimal path is still using the GPU for inference while CPU does MCTS**, just as the architecture intends [50]. So ensuring the GPU path is used (i.e. providing an `inference_fn` with `batch_inference` capability) is critical. If currently they are inadvertently on the slow path (e.g. using a Python lambda or something without batch_inference), that alone would explain being stuck at ~3k sims/sec – the code would be doing one forward pass per simulation in Python (~1k sims/sec per core as noted) [10] [11].

- **Inference callback overhead:** Even with batching, the callback mechanism has costs. `PyBatchInferenceCallback` calls into Python holding the GIL, which can create a hiccup if other Python threads try to log or do something at the same time (though ideally all sim threads are in C++ with GIL released). The conversion of game states to network input is done in pure Python (`state.get_enhanced_tensor_representation()` for each state, then `np.asarray` and stacking them [9]). This can likely be optimized. Each state could directly provide a ready-to-use tensor (perhaps via a C++->PyTorch DLPack interface). Right now, **there's redundant copying**: the state exists in C++ memory, then `get_enhanced_tensor_representation` might build a Python object (maybe a list or numpy array), then `np.asarray` makes it into a numpy array (copy), then that is sent to the GPU. The spec explicitly calls out removing these copies by exposing **zero-copy tensor handles via DLPack** [51]. By doing so, the C++ code could directly create a torch Tensor on pinned memory for the state, avoiding Python overhead for each state in the batch.

- **Throughput vs. latency trade-off:** The user's 30k sims/sec target assumes a certain balance of network latency and search computation. If the net is too large (making each inference slow), no amount of threading will hit 30k sims/sec because the GPU becomes the bottleneck. For example, if one inference batch of 32 takes 5 ms on the GPU, the absolute max if fully pipelined is 32 * (1000/5) = 6,400 sims/sec (plus overhead). To reach 30k, the network forward pass must be very fast (or the batch very large to amortize overhead). Given the hardware, a realistic assumption is a smaller network (perhaps a 5-10 layer CNN for Gomoku), which might achieve ~0.5–1 ms per 32-state batch. Combined with CPU parallelism, 30k is plausible. But if they try 19x19 Go with a large model, **30k is unrealistic on this GPU** – the network would dominate. So we must ensure the network complexity is aligned with throughput goals.

Comprehensive Plan to Achieve ~30k Simulations/Sec (or the Best Possible)

To drastically improve throughput, **major optimizations and some redesign are needed**. Below is a concrete, **step-by-step plan** addressing each issue above. These recommendations don't shy away from deep changes – if hitting >30k sims/sec requires it, it's on the table (short of switching to an entirely different framework or GPU-based tree search, which you ruled out). We focus on maximizing CPU usage and minimizing Python overhead while keeping the neural network flexible in Python.

A. Minimize Python's Role – Push More into C++

Goal: Let C++ handle all per-simulation work; Python should only handle high-level coordination and final results.

- **Move State Handling into C++:** Wherever possible, avoid calling back into Python for game state operations during search. The good news is your `alphazero::games::gomoku::GomokuState` is already C++. Ensure that the entire selection and expansion phase uses only C++ calls. In the current `SimulationRunner.select_leaf`, it already calls `current_state.isTerminal()` and `makeMove()` in C++ [52] [53]. That's good. But make sure Python isn't doing extra work like maintaining Python-side move mappings or cloning states outside C++. Ideally, the **state cloning should also happen in C++**. In `run_simulation` you do `root_state.clone()` in C++ at the start [54], which is perfect. In `ContinuousSimulationRunner`, you similarly clone in C++ (calling

`root_state.clone()` via pybind [55]. So state cloning is already native. **Avoid any Python cloning of state** (the old code had Python clones; that should be gone now). If any Python logic remains in the search loop (e.g. for Dirichlet noise injection at root or logging), consider moving it: e.g. apply Dirichlet noise inside C++ right after root expansion to keep that initial step GIL-free. The less Python does during search, the better.

- **Eliminate Python loop overhead:** The sync mode in `AlphaZeroMCTS.search` (which loops in Python calling `simulation_runner.run_simulation()` repeatedly) must be avoided for high performance. Always use the async `ContinuousSimulationRunner` mode with multiple threads. In fact, consider removing the Python-loop code path entirely in production builds to avoid accidental use. The C++ continuous loop is far more efficient (one C++ call that runs many simulations with GIL released) [56] [57]. This ensures only 1–2 GIL acquisitions per simulation (for inference) instead of potentially dozens if Python were looping [58] [59]. Your default `use_async_inference=True` and `parallel_mode="shared"` reflect this intent – stick to it.

- **Batch neural net calls in C++ (avoid Python per-state):** It's critical that the **fast batch inference path is used**. The engine already checks for `inference_fn.batch_inference`. Make sure your `inference_fn` is indeed an instance of `GPUInferenceWorker` wrapped by `CppInferenceBridge` so that `batch_inference` is available [60] [61]. This gives you the Mode 1 "Direct GPU Batching" where one Python call handles an entire batch [62] [63]. **Never use the legacy per-state inference** in production (that yields ~1k sims/sec as warned) [10] [11]. If currently you're seeing ~3000 sims/sec, it implies you might be partially using batch mode, but possibly with overhead. Double-check that the log message "Using direct GPU batch inference (fast path)" appears [64] [65]. If not, you may be inadvertently on the slow path and simply enabling the batch API could multiply performance by 10×.

- **Zero-copy state to tensor:** Implement a more direct pipeline from game state to neural network input. Right now, the Python batch callback builds a list of numpy arrays for positions [9], which then get converted to a PyTorch tensor internally. We can **cut out these copies**. One approach: use **DLPack** or a C++ extension to share memory. For example, allocate a contiguous buffer for the batch of state features in C++ (in pinned host memory). Have each `IGameState` write its features directly into this buffer (since the game state is C++, you can add a method like `fill_tensor(float* dest)` that writes the board planes). Then create a torch::Tensor from that buffer without copying (PyTorch can wrap a buffer via `torch::from_blob` or DLPack). By doing this inside the `PyBatchInferenceCallback` (which is C++ calling into Python), you could actually avoid Python iteration over states entirely: pass a capsule or DLPack tensor to Python which the `GPUInferenceWorker` can accept. The spec explicitly suggests *"expose zero-copy tensor handles (e.g., via DLPack) from C++ to Python callback to avoid np.array reallocations"* [66]. Implementing this will cut down on Python overhead **per batch** significantly. It turns the inference callback into essentially one GIL acquisition + one PyTorch call, rather than a Python-level loop over states.

- **Use PyTorch JIT or lighter models (optional):** If flexibility allows, consider using TorchScript for the model to reduce Python overhead further. You decided not to use libtorch (C++ API) for flexibility, which is fine. But you can still **JIT-compile the model in Python** (script or trace it) so that inference is just a matter of calling a compiled function. This might save some Python overhead inside `GPUInferenceWorker` itself (though likely minor if it's just forwarding to a model). Another trick: ensure `GPUInferenceWorker` is using `with torch.no_grad()` and **autocast (fp16)** around

inference. Mixed precision can give a big speedup on 3060 Ti (the guide expects it [50]). If not already, wrap the model call in `torch.cuda.amp.autocast()` to use FP16 – this can increase throughput ~30% for neural net forward passes.

- **Summary:** By pushing all simulation logic into C++ and streamlining the Python↔Torch interface, we reduce Python's contribution to a minimum. The engine should approach the ideal where Python only triggers the search and then consumes the final visit counts, doing virtually *zero work per simulation*. The spec's goal of *"remove per-search thread restarts and redundant tensor copies while keeping NN in Python"* captures this [67]. We must achieve exactly that.

B. Overhaul the Async Queue and Thread Coordination

Goal: Make simulation threads and the inference coordinator communicate with near-zero overhead, so CPU time is spent on MCTS, not waiting.

- **Use condition variables or futures instead of polling:** The current busy-wait loop should be replaced with a **blocking notification** mechanism. For example, the `AsyncInferenceQueue` can be implemented as a bounded buffer with a condition variable (or semaphore) that threads wait on when empty/full. Instead of each simulation thread sleeping 50µs and checking `queue.pending_count()`, they should **block on a cond-var and wake only when results are available or queue space frees up** [68] [69]. The BatchInferenceCoordinator thread can `notify_all` simulation threads when it posts new results, and simulation threads can `wait()` on a condition when they have nothing to do. Given the design, it might be easier to invert control: simulation threads block when trying to submit if the queue is full (std::condition_variable on `not_full`), and block waiting for results if they have nothing else to do (though ideally they always have another simulation to start). The spec explicitly says *"replace mutex-heavy producer/consumer with efficient wait/notify pipeline"* [68] – implementing this will eliminate the need for any artificial sleep. This can reclaim a lot of CPU cycles. It will also reduce context-switch thrashing (many threads waking frequently). Expect a noticeable boost in throughput, especially as thread count increases.

- **Redesign `PendingExpansion` storage:** The use of an `unordered_map` for pending expansions is a clear target for optimization. We can avoid dynamic lookups by using a **fixed-size array or pool**. For example, maintain a fixed array indexed by `request_id mod N` (N can be something like 8192 which is > max in-flight). Or, since each thread submits at most one new request at a time in the loop, you could incorporate the thread id into the request id (e.g., make `request_id = (thread_id << 32) | local_counter`). Then each thread can directly index into a vector of `PendingExpansion` for that thread. A simpler approach: **use the node index or pointer as key** – since each leaf can only be expanded once, you might even use the leaf node index as an identifier to match results. In fact, the coordinator returns `result.request_id` along with the node id (leaf) presumably [70]. If we trust node indices are unique per search, we might map results by node index (e.g., store pending expansions in an array where index = leaf node index). However, node indices are dense and large (millions), so that's not memory-efficient. Instead, consider reusing the **vector of paths** approach: you already allocate a `path_buffer_` per runner. You could allocate a corresponding array of `PendingExpansion` of size equal to max concurrent expansions (maybe a few thousand). Use a simple **ID counter that increments and wraps** (the ring buffer concept). The coordinator can then manage a ring of results. In any case, removing the hash map will save a lot of CPU. The plan in the spec is to *"consolidate PendingExpansion storage (pool +*

fixed-size arrays) to eliminate per-result map lookups and reversals" [68] [71] – exactly what we need. Implementing this means each result is retrieved in O(1) time and we can eliminate the explicit path reversal by storing the path in the needed order or storing the value to back up separately. This will significantly cut the overhead per simulation.

- **Batch process results more efficiently:** Currently, `process_completed_results` pops *all* ready results in a loop (which could be one at a time) [72]. Instead, process results in larger chunks to amortize overhead. If using a cond-var, the coordinator can wake up a simulation thread **only when a batch of results is ready**. The simulation threads could each check for results meant for them, or you could dedicate one thread to handle all results expansion (though that might become a bottleneck). A good approach is what the spec suggests: *the coordinator uses a wait/notify with a timeout rather than polling* [68] [73], and possibly posts results in batches. For instance, the coordinator could gather up to B results, then lock a mutex once and add all B to a result list, then notify. This way, simulation threads contend less on whatever synchronization is used to grab results. If each simulation thread currently calls `get_nowait()` in a tight loop [74] (as the Python pseudo-code in the guide did), that should be refactored to an event-driven approach where the coordinator pushes results to specific threads or a global list.

- **Tune the in-flight limits:** You have `kMaxInFlight = 4096` in the code as a cap on pending requests [75]. This is good to prevent unlimited growth, but it's quite high. If you truly had 4096 neural requests pending, that's an enormous backlog (and likely GPU would be saturated). In practice, with 12 threads, each thread might at most hold a few pending expansions before processing catches up. Consider if a smaller cap (e.g. 512) could reduce memory usage and map size (if that was an issue). But this is secondary – main thing is to ensure that cap doesn't throttle unnecessarily. If the GPU is fast, you might rarely hit 4096 anyway.

- **Thread-safety and data races:** Introducing more fine-grained locking or lock-free structures needs careful testing. The spec's risk section notes to use TSan (ThreadSanitizer) to verify no race conditions [76]. When rewriting the queue, use atomic operations or locks intentionally and keep them minimal.

Expected impact: A properly implemented wait/notify queue with O(1) pending lookup will drastically reduce the CPU wasted on coordination. The spec expects **async coordination overhead to drop below 20% of runtime** [77] (currently it's ~67% [2]). This could potentially double or triple effective simulations/sec, assuming inference is not yet saturated. It will especially improve scaling to more threads, as the overhead per thread goes down.

C. Optimize Multi-Threading and Parallel Efficiency

Goal: Use all 12 cores effectively, avoid any artificial serialization, and consider alternative threading models if needed.

- **Enable true multi-threading in one search:** Ensure that when you run a single search (say, for one move), **all desired threads are actually being utilized in parallel**. The old issue was that multi-threading was only used across separate searches, not within one search [78] [79]. The current code *does* distribute simulations across threads in one search (via ThreadPoolExecutor) [80] [81]. Make sure this path is always taken (i.e. `num_threads` is set to 12 or 24 as per the CPU). If anywhere it falls

back to a single-thread loop, fix that. The Python log should indicate multi-threaded search ("Multi-threaded search: N threads, ..." as in the code) [80] [82]. This seems fine now – just highlighting that enabling all cores is step one.

- **Tackle root expansion bottleneck:** The first expansion (root node) is a critical serial step. One idea is to **pre-expand the root prior to starting all threads**, so they have children to explore immediately. The code does attempt a "root pre-expansion" when adding Dirichlet noise [83], but then it comments "Do NOT pre-expand root – let C++ runner handle it" [84]. Perhaps due to a bug or simplicity, they leave it to the C++ thread. This means in every search, 11 out of 12 threads sit idle until the first network evaluation completes. To improve this, you could *speculatively expand the root on the main thread* (Python side) before launching the ThreadPool. The steps: evaluate the root state with the NN (batch of 1) to get prior and value, add children to the tree, then start the threads which will all find an expanded root and can immediately begin exploring different children. This removes that initial holdup. It costs one extra network eval at the beginning, but that is negligible amortized over many simulations (especially if simulations >> 1). Many implementations do exactly this (expand root and add Dirichlet noise before search). If there was a bug preventing it, fix that rather than forgoing the optimization. This change could shave the first few milliseconds off each search and improve multi-thread throughput (particularly noticeable for small simulation counts).

- **Reduce contention on node expansion:** The suggestion of per-thread node "arenas" is promising [38]. Implementation: give each thread its own block of node indices to allocate from, e.g. thread 0 gets indices 0–999k, thread 1 gets 1000k–1999k, etc. Then `allocate_nodes(n)` can often be satisfied by the thread's local block without an atomic operation (just a thread-local counter). Only when a block is exhausted, grab a new block from a global pool (rare). This will **greatly reduce atomic increments on the global node counter**. Since node allocation happens on every expansion, this could improve scaling. If implementing that is complex, a simpler partial fix: use larger atomic increments. E.g., have each thread atomically add a chunk (like 100 or 1000) to `num_nodes` and use that chunk for itself. This amortizes the atomic cost over many node allocations. It might slightly over-reserve nodes but that's okay if within bounds. Also ensure `MCTSTree.clear()` resets the per-thread allocators or global counter properly.

- **Tune virtual loss usage:** Virtual loss is essential for correctness, but its magnitude and usage can be tuned. You use `virtual_loss=1.0` by default [85], which is standard. Check if virtual loss is applied and removed correctly (looks like you handle it in selection and backup). One thing: you apply virtual loss to *children* as soon as they are selected [86] [87]. That's okay because it prevents other threads from selecting the same child. Just ensure that if a thread had to backtrack (like leaf already expanding), you remove the virtual losses along that path (`release_virtual_loss(path)` is called on backoff) [88] [89]. This appears handled. So not a problem – just a note that correctness is maintained.

- **Consider alternate parallelization strategies:** If after all these changes you still can't hit the target, you might consider more radical approaches, like *root parallelism* (each thread runs its own MCTS with fewer simulations and you average the results). This avoids shared contention entirely, but as noted in your guide, it suffers from the "stale frontier" problem – threads don't share information until the end [90]. The spec lists "experimental thread-local subtrees" as an option to benchmark [91]. Those tests showed much lower performance (~0.7k sims/sec for thread-local vs 3.5k for shared) [92], likely because lack of information sharing hurts efficiency per simulation. So, **shared-tree with**

proper locking is still the best for strength and good throughput. Stick to it, but it's worth validating through profiling that our locking isn't the limiter. If it were, then as a fallback, root parallelism could scale almost linearly with threads (since no interaction), but you'd need say 10 times more simulations to get equivalent result quality, which defeats the purpose. So we prefer to fix shared-tree rather than abandon it.

- **Thread affinity (advanced):** If after all main fixes, you're chasing the last 10-15% performance, consider pinning threads to cores (especially on Windows or Linux with pthreads). You could do this by retrieving the OS thread handle in C++ after ThreadPoolExecutor spawns them (this might require platform-specific API). Pin e.g. thread 0–5 to CCD0 cores and 6–11 to CCD1. This could improve cache usage. It's an advanced optimization with marginal gains, so only attempt once other issues are solved, and profile to see if it helps.

Expected impact: With better thread coordination and less waiting, you should see close to linear scaling up to ~8-12 threads. The target was a 6× speedup at 8 threads [32] – aim for that. On 12 threads, perhaps ~8× or more. By eliminating needless stalls (like root wait) and making expansions lock-free, the CPU cores will all be crunching different parts of the tree concurrently. This is vital for reaching tens of thousands of simulations/sec.

D. Streamline Tree Operations & Memory Access

Goal: Reduce per-simulation overhead on the CPU side (selection, expansion, backup) so that each simulation runs faster, using less CPU time and memory bandwidth.

- **Optimize Selection (PUCT calculation):** The PUCT child selection is already vectorized with AVX2 in `PUCTSelector.select_child()` [93] [94]. Ensure that AVX2 path is actually being used: compile with `-mavx2` and check `PUCTSelector.is_avx2_supported() == true` in logs [95]. If not, you're falling back to scalar code. Assuming it is, there's not much more to gain algorithmically – it's already a single pass computing all children's scores, with expansions flagged as -∞ to avoid selection [96] [97]. However, the spec hints at minor improvements: *reuse scratch buffers and move masking/normalization into SIMD loops* [98] [99]. In your code, after getting the NN policy, you do a separate loop in `expand_node()` to mask illegal moves and normalize probabilities [100] [101]. You could fold this into vectorized operations as well, but since it's done once per expansion (not per selection), it's less critical. The selection function itself looks solid. One possible micro-optimization: **pre-compute** `sqrt(parent_visits)` **once** and pass it, which you actually do (exploration_term) [102]. Good. Another: **tweak FPU (First Play Urgency) handling** – you currently set Q = 0 for unvisited, then maybe add a fixed fpu_value if enabled [103]. Some implementations dynamically scale FPU to reduce value bias. That's more about search quality than speed, though.

- **Speed up Backup:** The backup phase walks the path and does atomic updates on each node [104] [105]. Using atomics is necessary for thread safety, but you might examine if all these atomics are needed. For instance, virtual loss is subtracted at backup [105] – since only one thread will remove the virtual loss it added, that's fine. Visit count and total value increments must be atomic since multiple threads could back up through the same node concurrently. One idea: if contention on root node's atomics becomes an issue (many threads backing up at once), you could maintain per-thread counters and aggregate periodically – but that would complicate selection, so probably not worth it. The spec suggests "relax atomics where safe" [98] [99] – for example, maybe it's safe to update sibling

nodes or leaf node stats without atomic if no other thread touches that leaf at the same time (since only one thread expands a given leaf). But it's tricky: once expanded, other threads could visit it. Overall, the current atomic backup is standard and likely okay. If profiling shows a lot of time in `__sync_fetch_and_add`, then consider switching to C++ `<atomic>` with memory_order_relaxed where appropriate (e.g., incrementing visit counts might not need sequential consistency). However, be careful with floating-point atomic updates – you might use atomic<float> with bitwise atomic operations or keep using gcc intrinsics as you did.

- **Partial Backpropagation (idea):** A more radical idea: during backup, alternate threads could backpropagate value *only partway* and then mark a node for another thread to continue. This is an attempt to parallelize the backup of a long path. But given path lengths aren't huge (game depth is at most ~100 in Gomoku), this is overkill and likely not needed. It's better to keep it simple.

- **Memory locality:** Make sure the SoA arrays are aligned (they are, 64-byte align for SIMD) [106] [107]. The critical data that every simulation touches are: node visit_counts, total_values, virtual_losses, flags, etc. Access to these is quite random as different parts of the tree are visited. One possible improvement is **pre-fetching**: in the selection loop, after computing the best child index, you might prefetch that child's data (or its children indices) into cache before the next simulation accesses them. This is advanced and might not yield much unless there's a pattern. Since the code already uses contiguous child storage (first_child_index + i) [108], a depth-first traversal will somewhat sequentially access memory when expanding siblings, which is good.

- **Avoid unnecessary work:** Ensure that if a node is terminal, you mark it so (you do: set flag terminal and do not expand) [109] [110], and that selection breaks on terminal nodes [52] [111]. That avoids any needless expansion on won positions. Also, you skip winner check for early moves in Gomoku (nice optimization to speed isTerminal) [112] [113]. These little things help reduce the per-simulation cost.

- **Instrumentation & profiling:** To see where time is spent, use the instrumentation you built. There are `ScopedMetric` timers around Selection, Expansion, Backup in the C++ code [114] [115], and presumably those aggregate stats. Enable instrumentation (`enable_instrumentation=True`) and after a large search, call `mcts.get_statistics()`. It provides breakdowns like queue wait time, selection time, etc. The spec created metrics to ensure each part's contribution. Use this data to find the new bottleneck after implementing the above changes. For example, if after fixes, selection+backup is only 10% of time and inference is 50%, you know CPU-side is solved and GPU is the limiter, etc. This iterative profiling is crucial – "**Profile relentlessly: If you haven't measured it, it's wrong**" as your guide says [3].

E. Neural Network and GPU Optimizations

Goal: Ensure the neural network inference is not the limiting factor, and the CPU and GPU are balanced for max throughput.

- **Maximize GPU batch size & utilization:** The current async batching uses a minimum batch size of 32 and 2ms timeout. Monitor what batch sizes you're actually getting during a search. If the GPU is underutilized (say you see only 40-50% utilization), you can experiment with increasing `async_batch_size` to 48 or 64 and maybe a slightly longer timeout (e.g. 3–5ms) to allow larger batches to form [116] [50]. Larger batches improve GPU efficiency at the cost of a bit more latency.

Since your CPU can work in parallel, a slight latency increase is okay if it boosts throughput. The guide expected average batch 32–64 with 80–92% GPU usage [116]. Try to hit those figures. If batch sizes are too low because the CPU threads aren't producing work fast enough, then improving CPU (which we did above) will indirectly increase batch size as well (more leaves evaluated concurrently).

- **Use mixed precision (FP16) on GPU:** Ensure that in your `GPUInferenceWorker` or model code, you have enabled autocast. If not, add it. FP16 can nearly double inference throughput on GPUs that have tensor cores (like RTX 3060 Ti). The guide explicitly notes using mixed precision for 80%+ utilization [50]. Also use `torch.backends.cudnn.benchmark = True` if the input sizes are fixed, to let cuDNN choose the fastest kernel.

- **Streamline the inference pipeline:** Besides the zero-copy input, also consider pinning the output. The results (policy vector and value) are being passed back to C++ and then used to expand nodes. If the output is a numpy array or Python list currently (the code converts policy to list of floats [117] [118]), that's Python overhead. Instead, you could return a PyTorch tensor for the policy and a Python float for value (or a 1-element tensor for value) and let pybind11 convert the tensor to a buffer or even directly write it into C++ memory. Since you ultimately call `expand_node_with_result(node, state, policy_vector, value)` in C++ [119], perhaps we can eliminate converting the policy to a Python list and then back to C++ vector. One idea: have the batch inference callback return a list of `(np.ndarray, float)` *without converting to list of Python floats*. Your current code does `policy_array.tolist()` which is slow for large action spaces [117]. If you can change `PyBatchInferenceCallback` to accept a NumPy array directly, you might avoid that conversion – but pybind11 binding expects a `List[float]` for policy currently. To keep it simple, consider limiting the action space size: Gomoku has at most 225 moves – converting 225 floats to Python list isn't too bad. But for Go (361 moves) or Chess (~4672 moves if we encode underpromotion moves), that conversion could bite. The spec hints at "zero-copy path confirmed; fallback available" as a validation item [77], implying they plan to handle outputs more efficiently too. For now, focus on input side; output list conversion is a smaller issue.

- **Parallelize CPU inference (if GPU is bottleneck):** If after all this the GPU is maxed out but CPU threads are idle (unlikely, but suppose the net is huge), one could consider using CPU for some fraction of inferences to balance load. For example, run two inference workers: one on GPU, one on CPU, and send some requests to each. However, given GPU is much faster per evaluation and the code isn't set up for that, it's not an immediate recommendation. It's better to lighten the model or get a faster GPU if that's the case.

- **Evaluate network size vs. throughput:** Be realistic about what 30k sims/sec implies for the network. If you find that even after all optimizations, the GPU can't go beyond (say) 20k because each inference is too slow, consider simplifying the model architecture for the purpose of self-play speed. For example, use a smaller number of filters or layers. AlphaZero literature often used reduced models for smaller boards because beyond a point more sims were more valuable than a slightly stronger net per evaluation. You could also test on a simpler network (or even a dummy network that returns random policy) to see the upper bound of your MCTS framework without NN cost. That will tell you how many sims/sec the tree search *alone* can do (maybe it's tens of thousands). Then you know the remainder is NN cost.

Expected impact: Optimizing the GPU inference path will ensure that once the CPU side is improved, the GPU doesn't become the next bottleneck. Ideally, both CPU and GPU are fully utilized: CPU always has some simulations to run while GPU is crunching a batch. The outcome should be that the GPU runs at, say, 85%+ utilization (as measured by nvidia-smi) and the batch size distribution is healthy (most batches near the target size). This will support reaching the 30k sims/sec in practice, at least for games like Gomoku or small-board Go where the network is manageable.

F. Set Realistic Throughput Expectations

After all suggested improvements, let's assess what throughput is achievable on your hardware:

- **If all goes well:** The design target of **30–40k simulations/sec** with NN in loop is based on optimistic assumptions [1]. With a refined implementation, **30k sims/sec is potentially achievable** on Gomoku (15x15) or Tic-tac-toe level complexity using a moderately sized network. On a 5900X, we've seen similar projects reach ~20k sims/sec with 8 threads and a strong GPU. With 12 threads and further tuning, hitting the high 20k or low 30k range is conceivable. The spec sets **≥30k sims/sec for 1000 sims in Gomoku** as a target [32] – that indicates confidence it's reachable.

- **More complex games or larger nets:** If your target games include Chess or 19x19 Go, expect lower throughput. The state complexity and network size (especially for Go) will slow things down. Perhaps **10k–15k sims/sec** is more realistic for those on this hardware, even after optimizations. The network will simply take more time to evaluate. And high branching factor means more nodes expanded (which could actually increase sims/sec but also means more work per sim). So calibrate expectations per game. It's better to have 15k high-quality simulations in Chess than 30k low-quality or unrealistic target.

- **If 30k proves unrealistic:** You specifically asked for a realistic best-case figure. If after implementing these fixes you profile everything and see, for example, the GPU is the limiting factor at 20k sims/sec, then that is your cap unless hardware is upgraded. In that case, you might dial down the target and focus on **simulation quality**. Remember the guide's wisdom: *"30k intelligent simulations beat 100k dumb ones"* [120]. It's not just about the raw count – ensuring the MCTS and network are efficient in terms of decision quality matters too. So, if you top out at ~20k sims/sec with a larger net but the playing strength is good, that might be acceptable. The key is we don't want any *waste* in those 20k – which is why we remove Python overhead, etc.

- **Throughput vs. Strength trade-offs:** Some of the suggestions (like root parallelism, or reducing neural net size) involve trading some playing strength for more simulations. Apply these only if needed. The plan above mostly improves efficiency without hurting strength (it in fact **fixes** a prior strength bug that was cloning wrong state, if that was still present). We have been careful to maintain the fundamental algorithm: shared tree, virtual loss, etc., so the results should remain statistically the same (we're just doing it faster). Any change that could affect outcome (e.g. using floats vs doubles or relaxed atomics) should be tested to ensure no material impact on final move choices or win rates. The spec's success criteria include *policy/value parity ≤1e-4 vs baseline* [121] – keep that in mind when refactoring.

In summary, by **removing Python overhead, implementing a lock-efficient queue, better utilizing all CPU cores, reducing memory clears, and maximizing GPU batching**, we expect a **major throughput**

boost. The biggest gains will come from eliminating coordination inefficiencies (turning that 67% overhead into useful work) and from cutting out redundant Python data handling. After these changes, a realistic throughput on your Ryzen 5900X + RTX 3060 Ti could be on the order of **20k–30k simulations per second** for games like Gomoku. Hitting a stable 30k may require everything to line up perfectly (and the network to be small enough), but you will certainly move out of the paltry ~3k/sec range into tens of thousands.

Finally, remember to validate each optimization: use the built-in metrics and maybe write small microbenchmarks (e.g., time how long 1000 simulations take with a dummy network) to see the improvements step by step. The spec's plan was incremental with testing at each stage [122] – that's a good approach. By being **ruthlessly systematic** in addressing the slow points, you'll transform this MCTS engine from its current sluggish state into the high-throughput system it was meant to be.

Sources:

- Performance spec outlining current bottlenecks and targets [2] [32]
- MCTS code and design guide for implementation details [52] [16] [1]
- Code review highlighting Python loop and state-handling issues [4] [5]
- Spec tasks proposing concrete fixes (tree clearing, queue, zero-copy, etc.) [123] [21] [66]
- Internal documentation on threading problems (thread oversubscription) [31] and design principles [3].

---

[1] [3] [25] [28] [29] [30] [36] [46] [48] [49] [50] [74] [90] [106] [107] [116] [120] mcts_guide.md
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/mcts_guide.md

[2] [21] [22] [32] [38] [39] [51] [66] [67] [68] [69] [71] [73] [76] [77] [91] [92] [98] [99] [121] [122] [123] spec.md
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/specs/004-mcts-throughput-recovery/spec.md

[4] [5] [6] [7] [31] [37] [40] [41] [78] [79] mcts_review.md
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/mcts_review.md

[8] [9] [10] [11] [12] [13] [14] [33] [34] [35] [43] [44] [45] [47] [60] [61] [62] [63] [64] [65] [80] [81] [82] [83] [84] [85] [95] [117] [118] mcts.py
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/src/core/mcts.py

[15] continuous_simulation_runner.hpp
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/cpp_extensions/mcts/continuous_simulation_runner.hpp

[16] [17] [18] [19] [20] [23] [24] [26] [27] [42] [55] [70] [72] [75] [88] [89] [108] [119] continuous_simulation_runner.cpp
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/cpp_extensions/mcts/continuous_simulation_runner.cpp

[52] [53] [54] [86] [87] [100] [101] [104] [105] [109] [110] [111] [114] [115] simulation_runner.cpp
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/cpp_extensions/mcts/simulation_runner.cpp

[56] [57] python_bindings.cpp
https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/cpp_extensions/mcts/python_bindings.cpp

58 59 simulation_runner.hpp

https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/cpp_extensions/mcts/simulation_runner.hpp

93 94 96 97 102 103 selection.cpp

https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/cpp_extensions/mcts/selection.cpp

112 113 gomoku_state.cpp

https://github.com/cosmosapjw-quantum/omoknuni/blob/588699dd415e4ba40ef7fe0e85df7c4e10da15b9/cpp_extensions/games/gomoku/gomoku_state.cpp